# Lecture 07: Priority Queues in Practice

**Topics**

- Top-K patterns: finding top elements in massive datasets
- Event-driven simulation with priority queues
- Streaming algorithms: processing data you can't fit in memory
- Merge K sorted sequences efficiently
- Production patterns for real-world PQ applications

**Goals**

- Master the top-K pattern with bounded heaps
- Build an event simulator that processes actions in time order
- Handle streaming data with fixed memory budgets
- Understand when to use PQ vs sorting vs other structures

# Roadmap

**First half (≈45 min)**

- The top-K problem: naive vs PQ approaches
- Pattern: bounded min-heap for top-K tracking
- Streaming top tracks: process plays one at a time
- Complexity analysis: O(n log k) vs O(n log n)
- In-class exercise 1 (commit required)

**Break (3 min)**

**Second half (≈45 min)**

- Event-driven simulation: play queue with timestamps
- Merging K sorted streams: fan-in pattern
- Production patterns: tie-breaking, stability, pagination
- When NOT to use a PQ (and what to use instead)
- In-class exercise 2 (commit required)
- Complexity table and wrap-up

# Setup: load Spotify data

We'll use our familiar tracks and plays data throughout this lecture.

In [ ]:

```
import heapq
import csv
from collections import defaultdict, Counter

# Load tracks
tracks = {}
with open('data/tracks.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        tracks[row['track_id']] = row

# Load plays
plays = []
with open('data/plays.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        plays.append(row)

print(f"Loaded {len(tracks)} tracks and {len(plays)} plays")
print(f"Sample track: {list(tracks.values())[0]}")
print(f"Sample play: {plays[0]}")
```

# Part 1: The Top-K Problem

**Scenario:** You have millions of song plays. You need the **top 10 most-played tracks**.

**Naive approach:** Count all plays, sort all tracks, take the top 10.

- Time: O(n log n) for sorting all n tracks
- Space: O(n) to store all counts

**Better approach:** Use a bounded min-heap of size k=10.

- Time: O(n log k) — much better when k << n
- Space: O(k) — only keep k items in memory

**Key insight:** We don't need to sort everything. We just need the k largest items.

# Naive approach: sort everything

In [ ]:

```python
def top_k_naive(plays, k=10):
    """Find top-k most played tracks by sorting all tracks."""
    # Count plays per track
    play_counts = Counter(play['track_id'] for play in plays)

    # Sort ALL tracks by count (descending)
    sorted_tracks = sorted(play_counts.items(),
                           key=lambda x: x[1],
                           reverse=True)

    # Take top k
    return sorted_tracks[:k]

# Test it
top_10 = top_k_naive(plays, k=10)
print("Top 10 tracks (naive):")
for track_id, count in top_10:
    track_name = tracks[track_id]['name']
    print(f"  {count:3d} plays: {track_name}")
```

# Problem with naive approach

**What if we have 10 million tracks?**

Sorting 10M items: ~10M × $\log_2(10M)$ ≈ 230M comparisons

But we only wanted the top 10!

**Better idea:** Keep a **min-heap of size k**.

- Heap always contains the k largest items seen so far
- Minimum item in heap is the "k-th largest"
- New item beats the minimum? Pop minimum, push new item
- New item doesn't beat minimum? Ignore it

This gives us O(n log k) instead of O(n log n).

# Top-K with bounded min-heap

**Pattern:** Maintain a min-heap of size k. The minimum element in the heap is the k-th largest overall.

In [ ]:

```python
def top_k_heap(plays, k=10):
    """Find top-k most played tracks using bounded min-heap."""
    # Count plays per track
    play_counts = Counter(play['track_id'] for play in plays)

    # Use min-heap to keep top k items
    # Heap stores (count, track_id) tuples
    heap = []

    for track_id, count in play_counts.items():
        if len(heap) < k:
            # Heap not full yet, just add
            heapq.heappush(heap, (count, track_id))
        else:
            # Heap full: compare with minimum
            if count > heap[0][0]:  # heap[0] is minimum
                heapq.heapreplace(heap, (count, track_id))

    # Heap now contains top k items
    # Sort in descending order for display
    result = sorted(heap, reverse=True)
    return [(track_id, count) for count, track_id in result]
```

```python
# Test it
top_10_heap = top_k_heap(plays, k=10)
print("Top 10 tracks (heap):")
for track_id, count in top_10_heap:
    track_name = tracks[track_id]['name']
    print(f"  {count:3d} plays: {track_name}")
```

# Why min-heap for top-K?

**Counterintuitive:** We want the *largest* k items, but we use a *min*-heap.

**Why?**

- The heap contains the k largest items seen so far
- The *minimum* in the heap is the k-th largest overall
- That's the threshold: new items must beat it to get in
- Easy to check: just peek at `heap[0]`
- Easy to evict: `heappop()` removes the smallest (weakest link)

**Max-heap doesn't work** because we can't efficiently remove the smallest item.

**Visual:** Imagine the heap as the "VIP section" with k spots. The minimum is the "doorman" — beat them to get in.

# Complexity comparison

| Approach | Time | Space | When to use |
|---|---|---|---|
| Sort all | O(n log n) | O(n) | k ≈ n (need most items) |
| Bounded heap | O(n log k) | O(k) | k << n (small k) |
| `heapq.nlargest` | O(n log k) | O(k) | One-shot top-k |

**Example:** n = 10M tracks, k = 10

- Sort: 10M × $\log_2(10M)$ ≈ 230M comparisons
- Heap: 10M × $\log_2(10)$ ≈ 33M comparisons (7x faster!)

**Note:** Python's `heapq.nlargest(k, items, key=...)` uses this pattern internally.

# Using heapq.nlargest (production pattern)

In [ ]:

```python
def top_k_stdlib(plays, k=10):
    """Find top-k using heapq.nlargest (production style)."""
    play_counts = Counter(play['track_id'] for play in plays)

    # One line! heapq does the bounded heap internally
    top_items = heapq.nlargest(k, play_counts.items(),
                               key=lambda x: x[1])
    return top_items

# Test it
top_10_stdlib = top_k_stdlib(plays, k=10)
print("Top 10 tracks (stdlib):")
for track_id, count in top_10_stdlib:
    track_name = tracks[track_id]['name']
    print(f"  {count:3d} plays: {track_name}")
```

# Streaming top-K: process one item at a time

**Scenario:** Plays arrive in real-time. We can't wait to see all plays before computing top-k.

**Solution:** Maintain a bounded heap and update it as new plays arrive.

In [ ]:

```python
class StreamingTopK:
    """Track top-k items in a stream with bounded memory."""

    def __init__(self, k):
        self.k = k
        self.counts = defaultdict(int)
        self.heap = []  # Min-heap of (count, track_id)
        self.in_heap = set()  # Track which items are in heap

    def add_play(self, track_id):
        """Process a single play event."""
        self.counts[track_id] += 1
        new_count = self.counts[track_id]

        # If track is already in heap, we need to rebuild
        # (heapq doesn't support decrease-key directly)
        if track_id in self.in_heap:
            # Rebuild heap with updated counts
            self.heap = [(self.counts[tid], tid)
                         for tid in self.in_heap]
            heapq.heapify(self.heap)
        elif len(self.heap) < self.k:
```

```
            # Heap not full, add the track
            heapq.heappush(self.heap, (new_count, track_id))
            self.in_heap.add(track_id)
        elif new_count > self.heap[0][0]:
            # New count beats minimum in heap
            old_min = heapq.heapreplace(self.heap,
                                        (new_count, track_id))
            self.in_heap.remove(old_min[1])
            self.in_heap.add(track_id)

    def get_top_k(self):
        """Get current top-k items (sorted descending)."""
        result = sorted(self.heap, reverse=True)
        return [(track_id, count) for count, track_id in result]

# Simulate streaming plays
tracker = StreamingTopK(k=5)
for i, play in enumerate(plays):
    tracker.add_play(play['track_id'])

    # Show top-5 after every 100 plays
    if (i + 1) % 100 == 0:
        print(f"\nAfter {i+1} plays:")
        for track_id, count in tracker.get_top_k():
            print(f"  {count:3d} plays: {tracks[track_id]['name']}")
```

# Exercise 1: Top-K artists by total plays

**Task:** Find the top 5 artists by total play count.

**Requirements:**

1. Group plays by artist (use `tracks[track_id]['artist']`)
2. Use `heapq.nlargest` to find top 5 artists
3. Print artist name and total play count

**Commit required:** Commit your solution before the break.

In [ ]:
```
# YOUR CODE HERE
# Find top 5 artists by play count
```

# Break (3 minutes)

**Commit your Exercise 1 solution now!**

When we return:

- Event-driven simulation with timestamps
- Merging sorted streams
- Production PQ patterns

# Part 2: Event-Driven Simulation

**Scenario:** Model a music player's "Up Next" queue where songs are scheduled to play at specific times.

**Challenge:** Events don't arrive in time order — they're created when actions happen (user adds song, song finishes playing, skip button pressed).

**Solution:** Use a priority queue keyed by timestamp. Always process the earliest event next.

# Event simulation pattern

**Key idea:** Maintain a PQ of future events sorted by time. Process events in chronological order.

In [ ]:

```
class PlaybackSimulator:
    """Simulate music playback with scheduled events."""

    def __init__(self):
        self.events = []  # Min-heap of (timestamp, event_type, data)
        self.current_time = 0
        self.now_playing = None

    def schedule_event(self, timestamp, event_type, data):
        """Add an event to the queue."""
        heapq.heappush(self.events, (timestamp, event_type, data))

    def play_song(self, track_id, start_time):
        """Schedule a song to start playing."""
        duration = int(tracks[track_id]['duration_ms']) / 1000
        end_time = start_time + duration

        # Schedule start and end events
        self.schedule_event(start_time, 'START', track_id)
        self.schedule_event(end_time, 'END', track_id)

    def run_simulation(self, until_time=None):
        """Process events in chronological order."""
```

```
        print("Starting simulation...\n")

        while self.events:
            timestamp, event_type, data = heapq.heappop(self.events)

            # Stop if we've reached the time limit
            if until_time and timestamp > until_time:
                break

            self.current_time = timestamp

            if event_type == 'START':
                self.now_playing = data
                track = tracks[data]
                print(f"[{timestamp:7.1f}s] ▶️  START: {track['name']} '
                      f"by {track['artist']}")

            elif event_type == 'END':
                track = tracks[data]
                print(f"[{timestamp:7.1f}s] ⏹️  END:   {track['name']}"
                self.now_playing = None

        print(f"\nSimulation complete at t={self.current_time:.1f}s")

# Demo: schedule 3 songs
sim = PlaybackSimulator()
track_ids = list(tracks.keys())[:3]

# Songs start at different times (not in order!)
sim.play_song(track_ids[0], start_time=0)
sim.play_song(track_ids[1], start_time=195)  # After first song
```

```
sim.play_song(track_ids[2], start_time=50)   # Overlaps with first

sim.run_simulation(until_time=400)
```

# Why PQ for event simulation?

**Without PQ:** Sort all events upfront

- Can't handle dynamic events (new events created during simulation)
- Wastes time sorting far-future events we may never reach

**With PQ:** Events are processed as they become "current"

- New events can be added anytime: O(log n)
- Always get the next event in O(log n)
- Only sort what we actually process

**Real uses:**

- Discrete event simulation (networking, queueing theory)
- Game engines (process events in game-time order)
- Operating system schedulers (next task to run)
- Animation timelines (next keyframe/transition)

# Part 3: Merging K Sorted Sequences

**Scenario:** You have multiple sorted playlists (sorted by play count). You want to merge them into a single sorted stream.

**Naive approach:** Concatenate all lists, then sort: O(n log n)

**Better approach:** Use a min-heap to merge in O(n log k) where k is the number of lists.

# The K-way merge pattern

**Algorithm:**

1. Put the first element from each list into a min-heap
2. Pop the minimum element from the heap
3. Add the next element from that list to the heap
4. Repeat until all lists are exhausted

**Complexity:** O(n log k) where n = total elements, k = number of lists

**Key insight:** At any moment, we only need to compare k elements (one from each list), not all n.

In [ ]:

```python
def merge_k_sorted_lists(lists):
    """Merge k sorted lists into one sorted sequence."""
    result = []

    # Heap stores (value, list_index, element_index)
    # This lets us track which list to pull from next
    heap = []

    # Initialize heap with first element from each list
    for list_idx, lst in enumerate(lists):
        if lst:  # Skip empty lists
            heapq.heappush(heap, (lst[0], list_idx, 0))

    # Process elements in sorted order
    while heap:
        value, list_idx, elem_idx = heapq.heappop(heap)
        result.append(value)

        # Add next element from the same list
        next_idx = elem_idx + 1
        if next_idx < len(lists[list_idx]):
            next_val = lists[list_idx][next_idx]
            heapq.heappush(heap, (next_val, list_idx, next_idx))

    return result

# Demo: merge 3 sorted playlists
playlist_1 = [1, 4, 7, 10]    # Play counts from user 1's history
playlist_2 = [2, 5, 8, 11]    # User 2
playlist_3 = [3, 6, 9, 12]    # User 3
```

```
merged = merge_k_sorted_lists([playlist_1, playlist_2, playlist_3])
print("Merged play counts:", merged)
print("Is sorted?", merged == sorted(merged))
```

# Python's heapq.merge (production pattern)

**Good news:** You don't need to implement this yourself!

In [ ]:

```
# heapq.merge does exactly this
merged_stdlib = list(heapq.merge(playlist_1, playlist_2, playlist_3))
print("Using heapq.merge:", merged_stdlib)

# It even works with generators (streaming!)
def playlist_generator(items):
    for item in items:
        yield item

merged_streaming = list(heapq.merge(
    playlist_generator(playlist_1),
    playlist_generator(playlist_2),
    playlist_generator(playlist_3)
))
print("Streaming merge:", merged_streaming)
```

# Real-world use cases for K-way merge

| Scenario | What you're merging |
|---|---|
| Database query | Results from multiple shards/partitions |
| Log aggregation | Log files from multiple servers |
| Search engine | Results from multiple indices |
| External sort | Sorted chunks that didn't fit in memory |
| Time-series data | Streams from multiple sensors |

**Pattern:** Any time you have k sorted streams and need them combined in sorted order, use a PQ-based merge.

# Production PQ patterns: tie-breaking

**Problem:** What if two items have the same priority?

**Solution 1:** Add a tiebreaker field

In [ ]:

```python
# Bad: can fail if track_id isn't comparable with priority
# heapq.heappush(pq, (priority, track_id))

# Good: add a counter for stable ordering
counter = 0
pq = []
for track_id in ['track1', 'track2', 'track3']:
    priority = 5  # All same priority
    heapq.heappush(pq, (priority, counter, track_id))
    counter += 1

# Items come out in insertion order when priorities are equal
print("With tiebreaker:")
while pq:
    priority, cnt, track_id = heapq.heappop(pq)
    print(f"  {track_id} (priority={priority}, insertion={cnt})")
```

# When NOT to use a priority queue

| If you need... | Use this instead | Why |
|---|---|---|
| Sorted order for ALL items | Sort once: `sorted()` | O(n log n) once vs O(n log n) total for PQ |
| Random access by priority | Balanced BST, sorted list | PQ doesn't support lookup |
| Frequent priority updates | Fibonacci heap, indexed PQ | Standard heapq doesn't support decrease-key |
| Small k from large dataset | `heapq.nlargest/nsmallest` | Simpler, faster for one-shot queries |
| FIFO order | `collections.deque` | No priority needed |

**Use a PQ when:**

- You process items in priority order
- Items arrive dynamically (streaming)
- You only care about the min/max, not all items

# Exercise 2: Event-driven shuffle mode

**Task:** Implement a shuffle playlist simulator.

**Requirements:**

1. Take the first 5 tracks from the tracks dictionary
2. Assign each a random play time between 0 and 300 seconds
3. Use a priority queue to process them in time order
4. Print each track when it "plays" (in timestamp order)

**Hint:** Use `import random; random.uniform(0, 300)` for random times

**Commit required:** Commit your solution before class ends.

In [ ]:
```
import random

# YOUR CODE HERE
# Create shuffle simulator with random play times
```

# Complexity summary

| Pattern | Time | Space | Best for |
|---|---|---|---|
| Top-K with bounded heap | O(n log k) | O(k) | k << n |
| `heapq.nlargest/nsmallest` | O(n log k) | O(k) | One-shot queries |
| Event simulation | O(e log e) | O(e) | e events processed |
| K-way merge | O(n log k) | O(k) | n total items, k streams |
| Streaming top-K | O(n log k) per update | O(k) | Real-time tracking |

**Key insight:** When k << n, heap-based algorithms beat sorting by a huge margin.

# Wrap-up: PQ patterns in the wild

**You've learned:**

- ✅ Top-K pattern: bounded min-heap beats full sort when k << n
- ✅ Event-driven simulation: process events in chronological order
- ✅ Streaming algorithms: fixed memory budget for unbounded data
- ✅ K-way merge: combine sorted sequences efficiently
- ✅ Production patterns: tie-breaking, stability, choosing the right tool

**Next lecture:** Queues, stacks, and deques in applied pipelines

**Don't forget:** Commit both exercises before you leave!

# Complexity checkpoint

**Question 1:** You have 100M song plays. You want the top 100 songs. What's the best approach?

A) Sort all plays: O(100M log 100M)
B) Bounded heap: O(100M log 100)
C) Scan all, keep max: O(100M)
D) Binary search: O(log 100M)

**Question 2:** In event simulation, why use a PQ instead of sorting all events upfront?

A) PQ is always faster than sorting
B) New events can be added during simulation
C) PQ uses less memory
D) PQ gives better constant factors

**Answer:** B for both (bounded heap, dynamic events)